OpenZeppelin

Training for Constraining

How to Prevent Circuit-Level Bugs

Levi Katyal Sledd and Vijay Singh

50-minute workshop



https://github.com/V-k-h/training-for-constraining

Outline

Motivation

Circuits and Constraints

Kinds of Underconstraint Bugs

Deep Dive: gnark Fake GLV

Finding Underconstraints

Verification Infrastructure and AI

Wrap-Up

Motivation

Motivation: Underconstrained Circuits in 2026

System	Missing requirement	Reported consequence
Zcash Orchard	Binding of an ECC base-point cell	Counterfeit Orchard value
OpenVM pairing	Proper-subfield membership of a hint	Invalid pairing checks can pass
Scroll ecPairing	G2 subgroup membership	Sequencer/prover disagreement and finalization failure
ZKsync Airbender	Deferred constraints and verifier composition	Three criticals in the tooling assessment

Discovery does not establish prior exploitation. Detailed reports: appendix. Airbender: one critical circuit bug and two verifier-composition flaws.

Circuits and Constraints

Relations and Witnesses

Complex claims (e.g. zkVM execution, blockchain state transitions) can be expressed as knowing a secret witness that relates to public information.

Relations and Witnesses

Complex claims (e.g. zkVM execution, blockchain state transitions) can be expressed as knowing a secret witness that relates to public information.

Simple claim: "I know $w \in \mathbb{F}_{11}$ such that $x = w^4 + 5$."

Relations and Witnesses

Complex claims (e.g. zkVM execution, blockchain state transitions) can be expressed as knowing a secret witness that relates to public information.

Simple claim: "I know $w \in \mathbb{F}_{11}$ such that $x = w^4 + 5$."

- w: secret witness
- x: public witness
- Define $R(w, x)$ if $x = w^4 + 5$.

Relations and Witnesses

Complex claims (e.g. zkVM execution, blockchain state transitions) can be expressed as knowing a secret witness that relates to public information.

Simple claim: "I know $w \in \mathbb{F}_{11}$ such that $x = w^4 + 5$."

- w: secret witness
- x: public witness
- Define $R(w, x)$ if $x = w^4 + 5$.

Relations and Witnesses

Usually, the private witness and public witness are vectors:

- $w = (w_0, \ldots, w_k)$
- $x = (x_0, \ldots, x_\ell)$

Relations and Witnesses

Usually, the private witness and public witness are vectors:

- $w = (w_0, \ldots, w_k)$
- $x = (x_0, \ldots, x_\ell)$

The public witness can be public inputs, the expected output, or both.

Relations and Witnesses

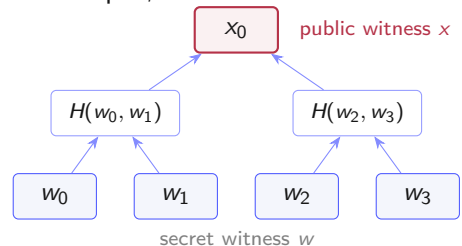
Usually, the private witness and public witness are vectors:

- $w = (w_0, \ldots, w_k)$
- $x = (x_0, \ldots, x_\ell)$

The public witness can be public inputs, the expected output, or both.

Applied example: $w = (w_0, w_1, w_2, w_3)$ are the secret leaves of a Merkle tree, $x = (x_0)$ is the public root, and

$$R(w, x): \quad H\big(H(w_0, w_1),\ H(w_2, w_3)\big) = x_0.$$



Relations and Witnesses

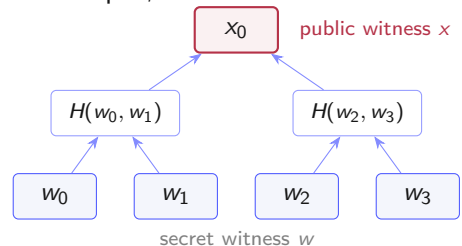
Usually, the private witness and public witness are vectors:

- $w = (w_0, \ldots, w_k)$
- $x = (x_0, \ldots, x_\ell)$

The public witness can be public inputs, the expected output, or both.

Applied example: $w = (w_0, w_1, w_2, w_3)$ are the secret leaves of a Merkle tree, $x = (x_0)$ is the public root, and

$$R(w, x): \quad H\big(H(w_0, w_1),\ H(w_2, w_3)\big) = x_0.$$



Note: we may use, e.g., s, t, u, v, w instead of $w_0, \ldots, w_3$ or x, y, z instead of x_0, x_1, x_2, as long as it's clear which are public and which are secret.

Arithmetic Circuits and Constraints

Arithmetic circuit: like a Boolean circuit, but

- wires carry values in $\mathbb{F}_p$.
- gates are $+$ and $\cdot \pmod{p}$.

Arithmetic Circuits and Constraints

Arithmetic circuit: like a Boolean circuit, but

- wires carry values in $\mathbb{F}_p$.
- gates are $+$ and $\cdot \pmod{p}$.

Arithmetic circuits $\equiv$ systems of *constraints*, one for each $+$ or $\cdot$ operation.

(There are other types of constraint systems, all equivalent).

Example: Constraint System and Witnesses

A constraint system $C = \{C_0, C_1, C_2, C_3\}$:

$C_0 : w_1 = w_0^2$
$C_1 : w_2 = w_1^2$
$C_2 : w_3 = w_2 + 5$
$C_3 : w_3 = x_0$

Example: Constraint System and Witnesses

A constraint system $C = \{C_0, C_1, C_2, C_3\}$:

$C_0 : w_1 = w_0^2$
$C_1 : w_2 = w_1^2$
$C_2 : w_3 = w_2 + 5$
$C_3 : w_3 = x_0$

Here,

- $x = (x_0)$ is the public witness.
- $w = (w_0)$ is the secret witness.

Example: Constraint System and Witnesses

A constraint system $C = \{C_0, C_1, C_2, C_3\}$:

$C_0 : w_1 = w_0^2$
$C_1 : w_2 = w_1^2$
$C_2 : w_3 = w_2 + 5$
$C_3 : w_3 = x_0$

Here,

- $x = (x_0)$ is the public witness.
- $w = (w_0)$ is the secret witness.
- (w_1, w_2, w_3) are *intermediate witnesses* (also secret).

Example: Constraint System and Witnesses

A constraint system $C = \{C_0, C_1, C_2, C_3\}$:

$C_0 : w_1 = w_0^2$
$C_1 : w_2 = w_1^2$
$C_2 : w_3 = w_2 + 5$
$C_3 : w_3 = x_0$

Here,

- $x = (x_0)$ is the public witness.
- $w = (w_0)$ is the secret witness.
- (w_1, w_2, w_3) are *intermediate witnesses* (also secret).
- $\overline{w} = (w_0, w_1, w_2, w_3)$ is the *extended* witness.

Example: Constraint System and Witnesses

A constraint system $C = \{C_0, C_1, C_2, C_3\}$:

$C_0 : w_1 = w_0^2$
$C_1 : w_2 = w_1^2$
$C_2 : w_3 = w_2 + 5$
$C_3 : w_3 = x_0$

Here,

- $x = (x_0)$ is the public witness.
- $w = (w_0)$ is the secret witness.
- (w_1, w_2, w_3) are *intermediate witnesses* (also secret).
- $\overline{w} = (w_0, w_1, w_2, w_3)$ is the *extended* witness.
- $(\overline{w}, x) \vDash C$ means that all constraints hold.

Example: Constraint System and Witnesses

A constraint system $C = \{C_0, C_1, C_2, C_3\}$:

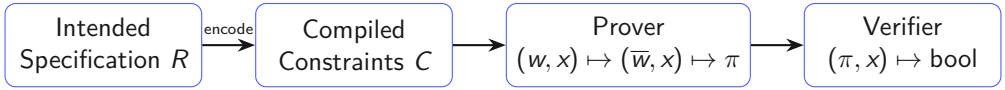
$C_0 : w_1 = w_0^2$
$C_1 : w_2 = w_1^2$
$C_2 : w_3 = w_2 + 5$
$C_3 : w_3 = x_0$

Here,

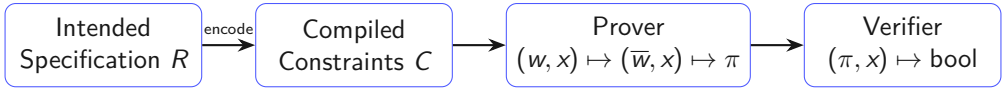
- $x = (x_0)$ is the public witness.
- $w = (w_0)$ is the secret witness.
- (w_1, w_2, w_3) are *intermediate witnesses* (also secret).
- $\overline{w} = (w_0, w_1, w_2, w_3)$ is the *extended* witness.
- $(\overline{w}, x) \vDash C$ means that all constraints hold.

In this case, $(w, x) \vDash C$ implies $x_0 = w_0^4 + 5$. Conversely, when $x_0 = w_0^4 + 5$ we can choose intermediate values satisfying C.

A SNARK Only Proves the Encoded Relation



A SNARK Only Proves the Encoded Relation

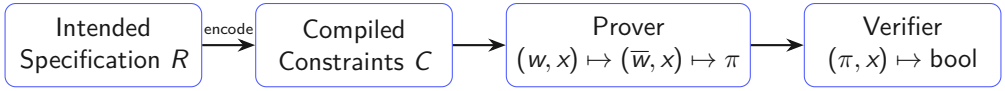


Ideally:

$$\boxed{(\overline{w}, x) \vDash C \quad \Longleftrightarrow \quad R(w, x)}$$

For a public-statement specification: $\exists w : (w, x) \vDash C \Leftrightarrow \exists w : R(w, x)$.

A SNARK Only Proves the Encoded Relation



Ideally:

$$\boxed{(\overline{w}, x) \vDash C \quad \Longleftrightarrow \quad R(w, x)}$$

For a public-statement specification: $\exists w : (w, x) \vDash C \Leftrightarrow \exists w : R(w, x)$.

An application can fail when one implication fails, even if the proof system works.

Hints

Honest computation

$$t \leftarrow w^2$$

The prover computes a *hint* outside the circuit: if honest, with a witness generator. The answer provided by the generator is called *advice*.

Enforced equation

$$t - w^2 = 0$$

A constraint forces every accepted witness to respect that computation.

The adversarial model

The prover may replace the witness generator, ignore advice, and substitute their own hints. Only the enforced constraints bind these choices.

Kinds of Underconstraint Bugs

Common Failure Patterns

Missing Requirement	What Can Go Wrong
Copy Constraint	Wires disconnect; values disagree.
Range Check	Overflow or underflow; unexpected values.
Hint Validation	Prover controls a computation.
Field / Group Membership Check	Algebraic properties are different outside of intended domain.
Public Witness Binding	Malleability; i/o can be unrelated to circuit.

Example 0: Missing Public Output Binding

Example 0: Missing Output Binding

Drop $C_3 : w_3 = x_0$ from our previous C, leaving

$C_0 : w_1 = w_0^2$
$C_1 : w_2 = w_1^2$
$C_2 : w_3 = w_2 + 5$

Example 0: Missing Output Binding

Drop $C_3 : w_3 = x_0$ from our previous C, leaving

$C_0 : w_1 = w_0^2$
$C_1 : w_2 = w_1^2$
$C_2 : w_3 = w_2 + 5$

False claim: "I know w_0 with $w_0^4 + 5 = 7$." No fourth power in $\mathbb{F}_{11}$ equals 2:

w_0	0	1	2	3	4	5	6	7	8	9	10
w_0^4	0	1	5	4	3	9	9	3	4	5	1

Example 0: Missing Output Binding

Drop $C_3 : w_3 = x_0$ from our previous C, leaving

$C_0 : w_1 = w_0^2$
$C_1 : w_2 = w_1^2$
$C_2 : w_3 = w_2 + 5$

False claim: "I know w_0 with $w_0^4 + 5 = 7$." No fourth power in $\mathbb{F}_{11}$ equals 2:

w_0	0	1	2	3	4	5	6	7	8	9	10
w_0^4	0	1	5	4	3	9	9	3	4	5	1

But $\overline{w} = (1, 1, 1, 6)$ and $x_0 = 7$ satisfy C_0, C_1, C_2: nothing ties w_3 to x_0.

Example 0: Missing Output Binding

Drop $C_3 : w_3 = x_0$ from our previous C, leaving

$C_0 : w_1 = w_0^2$
$C_1 : w_2 = w_1^2$
$C_2 : w_3 = w_2 + 5$

False claim: "I know w_0 with $w_0^4 + 5 = 7$." No fourth power in $\mathbb{F}_{11}$ equals 2:

w_0	0	1	2	3	4	5	6	7	8	9	10
w_0^4	0	1	5	4	3	9	9	3	4	5	1

But $\overline{w} = (1, 1, 1, 6)$ and $x_0 = 7$ satisfy C_0, C_1, C_2: nothing ties w_3 to x_0.

Repair: restore the copy constraint $C_3 : w_3 = x_0$, the only link between what the circuit computes and what the verifier sees.

Example 1: Unvalidated Hints

Example 1: An Unvalidated IsZero Hint

Let $w \in \mathbb{F}_{11}$ be a **secret witness** and $x \in \mathbb{F}_{11}$ a **public witness**.

$$R(w, x): \quad x = \mathsf{IsZero}(w) = \begin{cases} 1 & w = 0, \\ 0 & w \neq 0. \end{cases}$$

Example 1: An Unvalidated IsZero Hint

Let $w \in \mathbb{F}_{11}$ be a **secret witness** and $x \in \mathbb{F}_{11}$ a **public witness**.

$$R(w, x): \quad x = \mathsf{IsZero}(w) = \begin{cases} 1 & w = 0, \\ 0 & w \neq 0. \end{cases}$$

Let h be a secret hint. A witness generator advises

$$h = \begin{cases} w^{-1} & w \neq 0, \\ 0 & w = 0. \end{cases}$$

Example 1: An Unvalidated IsZero Hint

Let $w \in \mathbb{F}_{11}$ be a **secret witness** and $x \in \mathbb{F}_{11}$ a **public witness**.

$$R(w, x): \quad x = \mathsf{IsZero}(w) = \begin{cases} 1 & w = 0, \\ 0 & w \neq 0. \end{cases}$$

Let h be a secret hint. A witness generator advises

$$h = \begin{cases} w^{-1} & w \neq 0, \\ 0 & w = 0. \end{cases}$$

Suppose the constraints are just

$$C_0: \qquad x = 1 - wh.$$

Example 1: An Unvalidated IsZero Hint

Let $w \in \mathbb{F}_{11}$ be a **secret witness** and $x \in \mathbb{F}_{11}$ a **public witness**.

$$R(w, x): \quad x = \mathsf{IsZero}(w) = \begin{cases} 1 & w = 0, \\ 0 & w \neq 0. \end{cases}$$

Let h be a secret hint. A witness generator advises

$$h = \begin{cases} w^{-1} & w \neq 0, \\ 0 & w = 0. \end{cases}$$

Suppose the constraints are just

$$C_0: \qquad x = 1 - wh.$$

Attack: take $w = 3$, $h = 0$, $x = 1$. Constraint C_0 holds, but $\mathsf{IsZero}(3) = 1$ is false.

Repair: Constrain the Hint's Effect

Add

$$\boxed{C_1 : \ wx = 0.}$$

If $w \neq 0$, then $x = 0$, and $C_0 : x = 1 - wh$ then forces $h = w^{-1}$.

If $w = 0$, C_0 forces $x = 1$, and the value of h does not matter.

A free hint is fine when it cannot reach the public output: witness non-uniqueness is not underconstraint.

Repair: Constrain the Hint's Effect

Add

$$\boxed{C_1 : \ wx = 0.}$$

If $w \neq 0$, then $x = 0$, and $C_0 : x = 1 - wh$ then forces $h = w^{-1}$.

If $w = 0$, C_0 forces $x = 1$, and the value of h does not matter.

A free hint is fine when it cannot reach the public output: witness non-uniqueness is not underconstraint.

Lesson

A hint need not have a unique representation (may have multiple $\overline{w}$ for a given w). The *public behavior* must be pinned down by constraints.

Exercise 2: A Selector

Exercise 2: A Selector

Intended behavior: s selects x or y.

$$s \in \{0,1\}, \qquad z = \begin{cases} x & s = 1, \\ y & s = 0. \end{cases}$$

Implemented constraint over $\mathbb{F}_{11}$:

$$\boxed{z = sx + (1-s)y}$$

Here x, y are secret inputs. Fix public output $z = 5$. The selector s is secret.

Two minutes with your neighbour

Suppose $x = 3, y = 7$. Can you choose s so that the equation accepts?
Which constraint is missing?

Selector: False Proof and Repair

False Proof

For $s = 6$ in $\mathbb{F}_{11}$:

$$6 \cdot 3 + (1 - 6) \cdot 7 = 18 - 35 = -17 \equiv 5 \pmod{11}.$$

The implemented equation accepts, although $5 \notin \{3, 7\}$.

Selector: False Proof and Repair

False Proof

For $s = 6$ in $\mathbb{F}_{11}$:

$$6 \cdot 3 + (1 - 6) \cdot 7 = 18 - 35 = -17 \equiv 5 \pmod{11}.$$

The implemented equation accepts, although $5 \notin \{3, 7\}$.

Repair

$$\boxed{s(s-1) = 0} \qquad \text{and} \qquad z = sx + (1 - s)y.$$

In a field, the new equation forces $s = 0$ or $s = 1$.

Example 3: Range Checks and Foreign-Field Arithmetic

A Boolean Check Is a 1-Bit Range Check

$s(s-1) = 0 \Leftrightarrow s \in \{0, 1\}$: a range check on a single bit.

A Boolean Check Is a 1-Bit Range Check

$s(s-1) = 0 \Leftrightarrow s \in \{0, 1\}$: a range check on a single bit.

Field elements have no order and no sign. In $\mathbb{F}_{11}$, $3 - 5 = -2 = 9$.

A Boolean Check Is a 1-Bit Range Check

$s(s-1) = 0 \Leftrightarrow s \in \{0, 1\}$: a range check on a single bit.

Field elements have no order and no sign. In $\mathbb{F}_{11}$, $3 - 5 = -2 = 9$.

So claims like "balance $\geq$ withdrawal" need a *range check*: reading x as an integer in $\{0, \ldots, p-1\}$, show $x < 2^n$.

Range Checks by Bit Decomposition

Claim: $x < 4$ in $\mathbb{F}_{11}$. Secret witnesses are the bits w_0, w_1:

$C_0 : w_0(w_0 - 1) = 0$
$C_1 : w_1(w_1 - 1) = 0$
$C_2 : x = w_0 + 2w_1$

Range Checks by Bit Decomposition

Claim: $x < 4$ in $\mathbb{F}_{11}$. Secret witnesses are the bits w_0, w_1:

$C_0 : w_0(w_0 - 1) = 0$
$C_1 : w_1(w_1 - 1) = 0$
$C_2 : x = w_0 + 2w_1$
Then $x \in \{0, 1, 2, 3\}$, so $(w, x) \vDash C \Leftrightarrow R(x) : x < 4$.

Range Checks by Bit Decomposition

Claim: $x < 4$ in $\mathbb{F}_{11}$. Secret witnesses are the bits w_0, w_1:

$C_0 : w_0(w_0 - 1) = 0$
$C_1 : w_1(w_1 - 1) = 0$
$C_2 : x = w_0 + 2w_1$
Then $x \in \{0, 1, 2, 3\}$, so $(w, x) \vDash C \Leftrightarrow R(x) : x < 4$.

Pitfall: this needs $2^n \leq p$. With 4 bits in $\mathbb{F}_{11}$, $x = w_0 + 2w_1 + 4w_2 + 8w_3$ and

$$3 = 1 + 2, \qquad 3 = 2 + 4 + 8 - 11.$$

Two bit patterns, $(1, 1, 0, 0)$ and $(0, 1, 1, 1)$, for the same x. If the circuit reads w_3 as "$x \geq 8$", the prover can claim $3 \geq 8$.

Foreign-Field Arithmetic

Sometimes a circuit over $\mathbb{F}_p$ must check arithmetic modulo some $\ell \neq p$ (e.g. elliptic-curve scalars).

Foreign-Field Arithmetic

Sometimes a circuit over $\mathbb{F}_p$ must check arithmetic modulo some $\ell \neq p$ (e.g. elliptic-curve scalars).

- Write each integer in base B, $a = a_0 + a_1 B + a_2 B^2 + \cdots$, and range-check every *limb* $a_i < B$.

Foreign-Field Arithmetic

Sometimes a circuit over $\mathbb{F}_p$ must check arithmetic modulo some $\ell \neq p$ (e.g. elliptic-curve scalars).

- Write each integer in base B, $a = a_0 + a_1 B + a_2 B^2 + \cdots$, and range-check every *limb* $a_i < B$.
- Choose B small enough that limb products and carries stay below p, so native equations hold over the integers.

Foreign-Field Arithmetic

Sometimes a circuit over $\mathbb{F}_p$ must check arithmetic modulo some $\ell \neq p$ (e.g. elliptic-curve scalars).

- Write each integer in base B, $a = a_0 + a_1 B + a_2 B^2 + \cdots$, and range-check every *limb* $a_i < B$.
- Choose B small enough that limb products and carries stay below p, so native equations hold over the integers.
- To show $ab \equiv c \pmod{\ell}$, the prover hints a quotient k and the circuit checks $ab = k\ell + c$, with k and c range-checked too.

An Unbounded Quotient Checks Nothing

Small example: native $\mathbb{F}_{11}$, foreign $\ell = 3$, public $x, y, z \in \{0, 1, 2\}$. Check $xy \equiv z \pmod 3$ with

$$C_0 : xy - z = 3k \quad \text{in } \mathbb{F}_{11}.$$

An Unbounded Quotient Checks Nothing

Small example: native $\mathbb{F}_{11}$, foreign $\ell = 3$, public $x, y, z \in \{0, 1, 2\}$. Check $xy \equiv z \pmod 3$ with

$$C_0 : xy - z = 3k \quad \text{in } \mathbb{F}_{11}.$$

If k is unbounded, $3^{-1} = 4$ gives $k = 4(xy - z)$ for *any* x, y, z.

An Unbounded Quotient Checks Nothing

Small example: native $\mathbb{F}_{11}$, foreign $\ell = 3$, public $x, y, z \in \{0, 1, 2\}$. Check $xy \equiv z \pmod 3$ with

$$C_0 : xy - z = 3k \quad \text{in } \mathbb{F}_{11}.$$

If k is unbounded, $3^{-1} = 4$ gives $k = 4(xy - z)$ for *any* x, y, z.

Repair: $C_1 : k(k-1) = 0$. Then $|xy - z - 3k| < 11$, so C_0 holds over the integers.

An Unbounded Quotient Checks Nothing

Small example: native $\mathbb{F}_{11}$, foreign $\ell = 3$, public $x, y, z \in \{0, 1, 2\}$. Check $xy \equiv z \pmod 3$ with

$$C_0 : xy - z = 3k \quad \text{in } \mathbb{F}_{11}.$$

If k is unbounded, $3^{-1} = 4$ gives $k = 4(xy - z)$ for *any* x, y, z.

Repair: $C_1 : k(k-1) = 0$. Then $|xy - z - 3k| < 11$, so C_0 holds over the integers.

Lesson

Range-check the quotient *and* every other term, so the native equation cannot wrap. Missing this is the gnark fake-GLV bug.

Deep Dive: gnark Fake GLV

Fake GLV: Checking a Hinted Scalar Multiplication

Goal Establish $Q = [s]P$ with fewer curve operations. The prover supplies short scalars u, v and the candidate point Q.

$$\underbrace{[u]P = [v]Q,}_{\text{curve relation}} \qquad \underbrace{u \equiv sv \pmod{\ell}}_{\text{scalar relation}}.$$

If P, Q belong to a prime-order subgroup of order ℓ and $v \not\equiv 0 \pmod{\ell}$, then

$$[v](Q - [s]P) = O \quad \Longrightarrow \quad Q = [s]P.$$

Every hypothesis needs enforcement.

Two Fields Meet in One Gadget

Symbol	Meaning	Relevant arithmetic
r	Native circuit field modulus	Coordinates in $\mathbb{F}_r$
ℓ	Prime subgroup order	Scalars modulo ℓ

For BabyJubjub in a BN254 circuit, $r \neq \ell$.

$$u \equiv sv \pmod{\ell} \quad \text{is a relation in the scalar domain.}$$

But the equation is evaluated in $\mathbb{F}_r$.

A Free Quotient Makes the Check Vacuous

A simplified form of the vulnerable binding check:

$$\boxed{u - sv = k\ell \quad \text{in } \mathbb{F}_r}$$

Here k is a prover-supplied field element without sufficient bounds. Since $\ell \neq 0$ in $\mathbb{F}_r$,

$$k = (u - sv)\ell^{-1} \pmod{r}$$

exists for *every* choice of u, s, v.

The equation constrains k. It imposes no binding between u, s, and v.

The gnark Assertion Behind the Algebra

```
// Excerpt from the supplied v0.15.0 case study
s, _ := api.NewHint(halfGCD, 4, checkedScalar, curve.Order)
s1, s2, bit, k := s[0], s[1], s[2], s[3]
vs := api.Mul(s2, checkedScalar)
kL := api.Mul(k, curve.Order)
lhs := api.Select(bit, s1, api.Add(s1, vs))
rhs := api.Select(bit, api.Add(kL, vs), kL)
api.AssertIsEqual(lhs, rhs)
```

For a Boolean sign bit, the branches assert $u - sv = k\ell$ or $u + sv = k\ell$ in $\mathbb{F}_r$. Both branches let the free quotient absorb the discrepancy. Bounds on s_1, s_2 do not bind k.

Source: Source excerpt: supplied HTML case study. PR #1765 is its cited repair.

What a False Scalar Multiplication Means

Choose $u = 0$, $v = 1$, and $Q = O$.

$$[u]P = [v]Q = O.$$

The free quotient satisfies the native-field equation for any s. Thus the gadget can accept $Q = O$ even when $[s]P \neq O$.

Signature verification context

$$[S]B = R + [h]A, \qquad h = H(R, A, m).$$

If the gadget replaces $[h]A$ by O, the equation loses its intended key and message binding.

Fake GLV Repair: Restore Every Hypothesis

To infer $Q = [s]P$ from $[u]P = [v]Q$, require:

1. A **sound scalar congruence** $u \equiv sv \pmod{\ell}$ (proper foreign-field arithmetic or equivalent sound encoding).
2. P, Q in the intended **prime-order subgroup** of order ℓ.
3. $v \not\equiv 0 \pmod{\ell}$, so multiplication by v is invertible on that subgroup.
4. Sound constraints on signs, decomposition bounds, and the selected branches.

Then $[v](Q - [s]P) = O$ and cancellation gives $Q = [s]P$.

Crucial point

Bounding the quotient k *alone* does not establish the foreign-field relation. All terms need sound bounds/carry reasoning so native-field wraparound cannot invalidate the integer argument.

Source: gnark PR #1765 (verify precise affected paths and releases before attribution).

Finding Underconstraints

Manual Review Starts From the Specification

1. Write the intended domain and input/output relation.
2. Mark every public witness, secret witness, hint, and selector.
3. Trace the constraints that bind each required property.
4. Try a value outside the intended domain.
5. Add the missing constraint and retain the counterexample as a regression.

For the selector, the equation covered interpolation. The specification also required a Boolean domain.

Honest-Witness Tests Leave a Gap

Typical positive test

$$\overline{w} \leftarrow \mathrm{HonestWitness}(x), \qquad (\overline{w}, x) \vDash C.$$

Soundness question

$$\exists \overline{w}, x : \; C(\overline{w}, x) = 0 \;\wedge\; \neg R(w, x) \; ?$$

- Positive tests check examples of valid executions.
- Adversarial witness tests search outside the honest generator's outputs.
- Missing constraints can leave every positive test passing.

Random Search versus Structured Witnesses

Randomly changing only public inputs often runs the honest witness generator and misses underconstraints.

Mutate private witnesses and hints: change a selector, quotient, or candidate point while keeping other constraints satisfiable.

Structured attacks beat blind random search: if a quotient k is free, calculate $k = (u - sv)\ell^{-1}$ directly rather than guessing it.

Demo: Fuzz the Relation, Including the Witness

```
# F_11; bypass the honest selector generator
x, y = 3, 7
for _ in range(1000):
    b = rng.randrange(11)
    z = (b*x + (1-b)*y) % 11
    if accepts(x, y, z, b) and z not in (x, y):
        report_counterexample(x, y, z, b)
        break
```

Oracle: acceptance plus a violation of the independent specification.

Randomize hints and private wires. Preserve the constraints you want to test.

Formal Verification: Search for an Invalid Witness

For public witness x and extended secret witness $\overline{w}$, check satisfiability of

$$\boxed{\exists(\overline{w}, x) : \ (\overline{w}, x) \vDash C \ \wedge \ \neg R(w, x).}$$

- **SAT:** a witness-level counterexample to the specification exists.
- **UNSAT:** no counterexample exists *in the encoded model*.
- **UNKNOWN / timeout:** no verification conclusion.

For the selector: $z = sx + (1 - s)y$ and $z \notin \{x, y\}$ is satisfiable.

Formal Verification: Search for an Invalid Witness

For public witness x and extended secret witness $\overline{w}$, check satisfiability of

$$\boxed{\exists(\overline{w}, x) : \ (\overline{w}, x) \vDash C \ \wedge \ \neg R(w, x).}$$

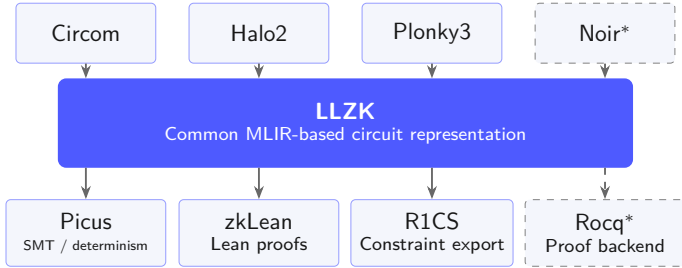
- **SAT:** a witness-level counterexample to the specification exists.
- **UNSAT:** no counterexample exists *in the encoded model*.
- **UNKNOWN / timeout:** no verification conclusion.

For the selector: $z = sx + (1 - s)y$ and $z \notin \{x, y\}$ is satisfiable.

Add $s(s - 1) = 0$: the query becomes unsatisfiable over every field.

Verification Infrastructure and AI

Formal Verification: Toward a Unified Framework



Why it matters

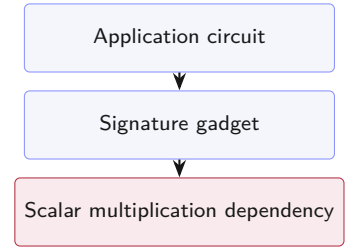
One intermediate representation enables reusable analyses across circuit languages; *LLZK is infrastructure, not itself a soundness proof.*

*Evolving integrations / research directions, not universal production support.

LLZK is open source, developed by Veridise with support from the Ethereum Foundation. https://github.com/project-llzk

Scalability note: Bit-blasting large-prime-field arithmetic into Boolean constraints can be costly. Generic SMT solvers may struggle with nonlinear field operations; specialized algebraic reasoning and circuit-aware verification backends help. LLZK provides the representation, not the solver.

Why AI-Assisted Review? The Dependency Problem



Why manual review alone is hard

- Security assumptions cross library and compiler boundaries.
- Advisories, versions, and reachable call paths change.
- Honest-witness tests can pass even when a dependency is underconstrained.

AI-assisted workflow

Find relevant advisories → **trace** affected call paths → **propose** malicious witnesses → **verify** using tests, solvers, or proofs.

AI broadens the search; executable evidence establishes the finding.

How We Review ZK Systems at OpenZeppelin

AI Auditor
Dependency tracing, hypotheses

MLIR / LLZK
Analyze circuit constraints

Lean proofs
Check mathematical formulas

Human auditors
Review assumptions and exploits

Complementary approaches

AI helps find places to look; formal methods establish selected properties; human reviewers connect the proofs and counterexamples to the real security model.

None alone establishes end-to-end security.

Wrap-Up

Questions

Which property does your circuit need to enforce?

Run the counterexamples yourself

```
git clone https://github.com/V-k-h/training-for-constraining
cd training-for-constraining
make demo # five counterexamples over F_11
make test # exhaustive checks of the repairs
make lean # optional: machine-checked theorems
```

Python 3.10+ only. Lean 4 and LLZK material is optional.



https://github.com/V-k-h/training-for-constraining

Appendix: Source Reports

- Shielded Labs: Orchard counterfeiting disclosure (June 4, 2026).
- Orchard missing-copy-constraint technical description.
- OpenVM: CVE-2026-46669 (May 15, 2026).
- Scroll Security Council: February 23 emergency upgrade report.
- Veridise: Airbender tooling report (April 22, 2026), pp. 3, 11–17.

Appendix: Fake GLV and Further Reading

- Supplied reference: `fake-glv-underconstraint.html`. Main workshop uses its algebraic example, with explicit subgroup assumptions.
- gnark PR #1765: repair cited by the supplied reference.
- Fake GLV: original method discussion (2024).
- ZKSecurity: AI-assisted OpenVM discovery (2026).

The fake-GLV production version and patch claims come from the supplied reference. Check the release advisory before presenting exact affected-version ranges.

Appendix: A Source-Linked Dependency Review Inventory

Published example	Review question
gnark signature malleability	Does the caller establish the required scalar range?
gnark range-check width	Does the enforced bound match the application's intended width?
gnark fake-GLV DoS	Can hostile inputs make witness generation stall? This is distinct from our soundness case.
OpenVM pairing soundness	Does the optimized check enforce the required subfield?

Illustrative inventory, not a complete issue list. Applicability requires version and call-path review.

Appendix: Why Short Scalars Help

$$u \equiv sv \pmod{\ell}, \qquad |u|, |v| \text{ roughly } \sqrt{\ell}.$$

The honest prover finds a short lattice vector, often with an extended-Euclidean or half-GCD method.

- Short scalars reduce the double-and-add work in $[u]P - [v]Q$.
- Finding the decomposition happens outside the circuit.
- The circuit still needs to enforce its algebraic relation, signs, bounds, and nonzero denominator.